ComicCraft

AI Comic Story Creator using Gemini Models

Project Report

Generative AI project: script writing and panel illustration with Google Gemini

Stack: Python, Flask, google-genai SDK, HTML/CSS/JavaScript

1. Abstract

ComicCraft is a web application that turns a short story idea into a finished comic page. The user describes a story, chooses a genre, tone, art style and number of panels, and the system does the rest. A Gemini text model writes a structured script with characters, captions and dialogue. A Gemini image model then draws every panel, using the first panel as a visual reference so that characters look the same from page to page.

The application is built with a Flask back end and a plain HTML, CSS and JavaScript front end. Speech bubbles and captions are rendered as editable page text instead of being baked into the images, which avoids the garbled lettering common in AI-generated art and lets the user correct any line before printing or saving the comic as a PDF.

2. Introduction and Problem Statement

Making a comic normally needs two skills at once: storytelling and illustration. Most people who have a good story idea cannot draw, and most artists still spend hours on layout and lettering. Generative AI can now write and draw, but using separate tools for each step produces mismatched results, and image models alone struggle to keep a character consistent across several images.

ComicCraft solves this by joining both steps in one guided workflow. The script generator decides what each panel shows and fixes each character's appearance in writing. The image generator receives that description on every call, plus a reference image, so the story reads as one comic and not as unrelated pictures.

3. Objectives

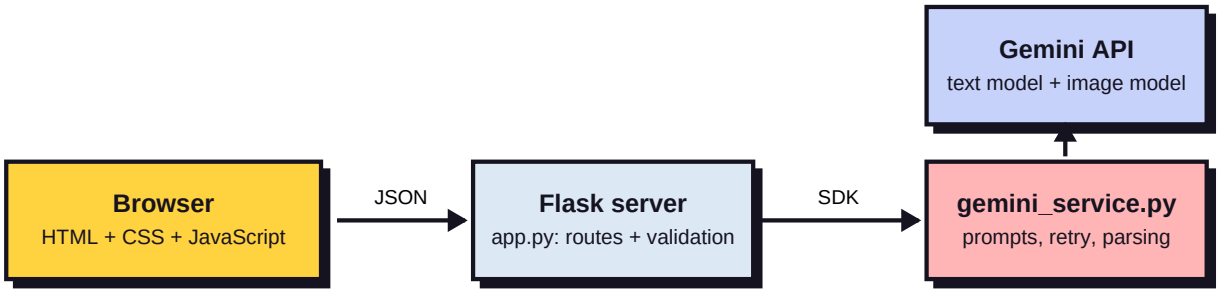
- Convert a free-text idea into a structured comic script in one request.
- Generate a coherent illustration for each panel in a chosen art style.
- Keep characters visually consistent across panels.
- Let the user edit text, redraw a panel, or change a panel's scene without starting over.
- Export the result as a printable PDF and as a JSON script.
- Handle API failures (rate limits, blocked images, bad keys) with clear messages instead of crashes.

4. Key Features

Feature	What it does
Story settings	Idea (10 to 1500 characters), 7 genres, 5 tones, 6 art styles, 3 to 8 panels.
Script generation	Gemini returns JSON: title, character list with fixed looks, and per-panel caption, dialogue and scene.
Panel illustration	One image per panel, generated one at a time so progress is visible and one failure does not lose the rest.
Character consistency	Character descriptions are sent with every image request, and panel 1 is passed in as a reference for panels 2 onward.
Live editing	Title, captions and speech bubbles can be edited directly on the page.
Redraw and edit scene	Any panel can be redrawn, or given a new scene description, without touching the others.
Export	Print or save as PDF from the browser, or download the script as a JSON file.

5. System Architecture

The system has three layers. The browser collects input and shows results. The Flask server validates every request and returns JSON. A dedicated service module holds all Gemini logic, so prompts, retries and response parsing live in one place and can be tested on their own.



Requests go left to right; images and script return the same way.

5.1 Project structure

```
comiccraft/
  app.py                 Flask routes and input validation
  gemini_service.py      Gemini prompts, retry logic, parsing
  requirements.txt       Python dependencies
  .env.example           API key and model settings template
  templates/index.html   Page layout
  static/style.css       Comic-style interface
  static/app.js          Front-end logic
  tests/test_app.py      Automated tests (no API key needed)
  .vscode/               Launch and test settings
```

6. Technology Stack

Layer	Technology	Why it was chosen
Language	Python 3.10+	Simple, and first-class Gemini SDK support.
Back end	Flask	Small, easy to run in VS Code, ideal for a few JSON endpoints.
AI SDK	google-genai	Google's current SDK for Gemini text and image generation.
Text model	gemini-2.5-flash (configurable)	Fast, follows JSON instructions well.
Image model	gemini-2.5-flash-image (configurable)	Generates images and accepts a reference image for consistency.
Front end	HTML, CSS, JavaScript	No build step, so the project opens and runs immediately.
Config	python-dotenv	Keeps the API key out of source code.
Testing	pytest	Fast tests with the Gemini calls mocked.

7. Working of the System

The workflow has two stages: writing, then drawing.

- **Write.** The browser sends the idea and settings to `POST /api/story`. The server checks the input, then asks the text model for a JSON script. The response is cleaned (code fences removed) and validated: panels are numbered, text is trimmed, and at most three dialogue lines are kept per panel.
- **Draw.** The browser builds empty panel cards, then calls `POST /api/panel` for each panel in order. Each call sends the scene, the character descriptions, the art style and, after panel 1, the first image as a reference.
- **Assemble.** Each returned image is placed in its card. Captions and speech bubbles are drawn on top as editable text.
- **Refine and export.** The user edits text, redraws panels as needed, then prints to PDF or downloads the script.

7.1 API endpoints

Endpoint	Method	Purpose
/	GET	Serves the web page.
/api/health	GET	Reports whether an API key is set and which models are configured.
/api/story	POST	Input: idea, genre, tone, style, panels. Output: title, characters, panels.
/api/panel	POST	Input: scene, characters, style, optional reference image. Output: image as a base64 data URL.

8. Prompt Design and Character Consistency

Good output depends on how the models are asked. ComicCraft uses four techniques.

- **Structured output.** The story request asks for one exact JSON shape and sets the response type to JSON, so the result can be parsed without guessing.

- **Written character sheets.** The script includes a fixed 'look' for every character (hair, clothes, colors). This text is repeated in every image prompt.
- **Reference image.** From panel 2 onward the first panel is sent alongside the prompt, with an instruction to match faces, outfits and art style.
- **No lettering in art.** Image prompts forbid text, letters and speech bubbles, because the page draws them itself as clean, editable text.

9. Error Handling and Reliability

Situation	How ComicCraft responds
Missing or invalid API key	Warning banner on the page; clear message on request.
Rate limit or temporary server error	Up to 3 attempts with a growing pause, then a plain 'wait a minute' message.
Model returns bad JSON	Fence stripping and brace extraction first; if that fails, asks the user to try again.
Image blocked or empty	That panel shows an error and a Redraw button; other panels are kept.
Invalid user input	Rejected with HTTP 400 and a specific message (short idea, panels outside 3 to 8).
Server unreachable	The browser shows 'Cannot reach the server' instead of failing silently.

10. Testing

The test suite runs ten checks and needs no API key because the Gemini functions are replaced with stand-ins. It covers the home page, the health endpoint, input validation, successful story and panel responses, error propagation as HTTP 502, story normalization, and JSON extraction from code-fenced text. Real end-to-end generation must be checked manually with a valid key.

Manual test checklist:

- Create a 3-panel comic and confirm all three images appear.
- Confirm the same character looks alike across panels.
- Edit a caption, then redraw a panel and confirm the edit is kept on other panels.
- Start the server with no key and confirm the warning banner appears.
- Use Print, choose Save as PDF, and confirm the panels are not cut across pages.

11. Setup and Running

```
1. Unzip the project and open the folder in VS Code.
2. python -m venv .venv   (then activate it)
3. pip install -r requirements.txt
4. Copy .env.example to .env and set GEMINI_API_KEY
5. python app.py          (or press F5 in VS Code)
6. Open http://127.0.0.1:5000
7. pytest                 (runs the tests)
```

A free API key can be created in Google AI Studio. Model names are set in `.env`, so they can be changed if Google renames or retires a model.

12. Limitations

- Character consistency is improved but not guaranteed; the image model can still vary faces or clothing.
- Panels are drawn one after another, so a comic takes roughly 10 seconds per panel or more on the free tier.
- Comics are not saved on the server. Users must export the PDF or JSON before closing the page.
- Image models may refuse some prompts, and the app asks for family-friendly content only.

13. Future Scope

- User accounts and a gallery to save and reopen comics.
- Choice of page layouts (strips, splash panels, varied panel sizes).
- Download the whole page as a single image or a multi-page PDF generated on the server.
- Upload a character photo or sketch to use as the reference for a main character.
- Multi-language scripts and text-to-speech narration.
- Streaming responses and parallel panel generation for faster results.

14. Conclusion

ComicCraft shows how two Gemini models can be combined into a practical creative tool. One model plans the story and fixes each character's look in text; the other turns that plan into artwork. Keeping the lettering as editable page text, drawing panels one at a time, and handling errors clearly make the result reliable enough for real use, while the small Flask and JavaScript code base stays easy to read, run and extend.